

Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI

Email: vtu29748@veltech.edu.in

Roll no: vtu29748

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 11_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 42.5

Section 1 : Coding

1. Problem Statement

You are given a bipartite graph represented using an edge list. The graph contains two sets of nodes tasks and processors. Each edge connects a task node to a processor node, indicating that the processor can execute that task.

Your goal is to determine the maximum matching in the bipartite graph. A matching is a set of edges such that:

Each task is assigned to at most one processor. Each processor is assigned to at most one task.

Write a program to compute the maximum number of task–processor assignments possible.

Input Format

The first line contains an integer T, representing the number of tasks.

The second line contains an integer P, representing the number of processors.

The third line contains an integer E, representing the number of edges in the bipartite graph.

The next E lines each contain two space-separated integers task and processor where this indicates that the given task can be assigned to the specified processor.

Output Format

The output prints the result in the following format: "The maximum matching is: X", where X is the maximum number of matches that can be made in the bipartite graph.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

3

5

1 1

2 2

4 3

3 3

4 1

Output: The maximum matching is: 3

Answer

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int T, P, E;
```

```
int graph[15][15];
```

```
int matchP[15];
```

```
int visited[15];
```

```
int bpm(int u) {
```

```

    for (int v = 1; v <= P; v++) {
        if (graph[u][v] && !visited[v]) {
            visited[v] = 1;
            if (matchP[v] == 0 || bpm(matchP[v])) {
                matchP[v] = u;
                return 1;
            }
        }
    }
    return 0;
}

int main() {
    scanf("%d", &T);
    scanf("%d", &P);
    scanf("%d", &E);

    memset(graph, 0, sizeof(graph));

    for (int i = 0; i < E; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;
    }

    memset(matchP, 0, sizeof(matchP));

    int result = 0;

    for (int u = 1; u <= T; u++) {
        memset(visited, 0, sizeof(visited));
        if (bpm(u))
            result++;
    }

    printf("The maximum matching is: %d", result);

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Irfan is working on a project that involves managing relationships between two sets of entities, set U and set V. He wants to determine the maximum matching in a bipartite graph to optimize connections without overlap. Irfan also wants to explore different scenarios by adding a new connection (edge) between two entities and observing how it affects the maximum matching.

Your task is to compute the size of the maximum matching before and after adding a specified edge.

Note: Vertices in set U are numbered from 0 to $n-1$, and vertices in set V are numbered from 1 to m .

Input Format

The first line contains an integer n , representing the number of vertices in set U.

The second line contains an integer m , representing the number of vertices in set V.

The third line contains an integer e , representing the number of edges.

The next e lines each contain two integers u and v , representing an edge between vertex u from set U and vertex v from set V.

The last line contains two integers u and v , representing the edge to be added.

Output Format

The output prints the results in the following format:

Size of maximum matching before adding edges: X

Size of maximum matching after adding edges: Y

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

4

3

0 1

0 2

1 1

1 2

Output: Size of maximum matching before adding edges: 1

Size of maximum matching after adding edges: 1

Answer

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int n, m, e;
```

```
int graph[105][105];
```

```
int matchV[105];
```

```
int visited[105];
```

```
int bpm(int u) {
```

```
    for (int v = 1; v <= m; v++) {
```

```
        if (graph[u][v] && !visited[v]) {
```

```
            visited[v] = 1;
```

```
            if (matchV[v] == -1 || bpm(matchV[v])) {
```

```
                matchV[v] = u;
```

```
                return 1;
```

```
            }
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

```
int maxMatching() {
```

```
    memset(matchV, -1, sizeof(matchV));
```

```
    int count = 0;
```

```
    for (int u = 0; u < n; u++) {
```

```
        memset(visited, 0, sizeof(visited));
```

```
        if (bpm(u))
```

```
            count++;
```

```
    }
```

```
    return count;
```

```
}
```

```

int main() {
    scanf("%d", &n);
    scanf("%d", &m);
    scanf("%d", &e);

    memset(graph, 0, sizeof(graph));

    for (int i = 0; i < e; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;
    }

    int before = maxMatching();

    int u, v;
    scanf("%d %d", &u, &v);
    graph[u][v] = 1;

    int after = maxMatching();

    printf("Size of maximum matching before adding edges: %d\n", before);
    printf("Size of maximum matching after adding edges: %d", after);

    return 0;
}

```

Status : Partially correct

Marks : 2.5/10

3. Problem Statement

Jack wants to create a program to manage a bipartite graph, which contains two sets of vertices. He needs the program to:

Initialize the graph with a specific size, Add edges between the two sets of vertices, and Find pairings where each vertex from the first set is matched with at most one vertex from the second set, and no two vertices from the first set are matched with the same vertex from the second set.

The goal is to determine the maximum matching pairs in the bipartite

graph.

Input Format

The first line contains an integer m , representing the number of vertices in the first set.

The second line contains an integer n , representing the number of vertices in the second set.

The third line contains an integer e , representing the number of edges in the graph.

The next e lines each contain two integers u and v , representing an edge between vertex u in the first set and vertex v in the second set.

Output Format

The output prints the maximum matching pairs in increasing order of u , where each pair is printed in the format: (u, v)

If there are no matching pairs, print: "No matching found"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

3

5

1 1

2 2

4 3

3 3

4 1

Output: (1, 1)

(2, 2)

(3, 3)

Answer

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int m, n, e;  
int graph[15][15];  
int matchR[15];  
int visited[15];
```

```
int bpm(int u) {  
    for (int v = 1; v <= n; v++) {  
        if (graph[u][v] && !visited[v]) {  
            visited[v] = 1;  
            if (matchR[v] == 0 || bpm(matchR[v])) {  
                matchR[v] = u;  
                return 1;  
            }  
        }  
    }  
    return 0;  
}
```

```
int main() {  
    scanf("%d", &m);  
    scanf("%d", &n);  
    scanf("%d", &e);
```

```
    memset(graph, 0, sizeof(graph));
```

```
    for (int i = 0; i < e; i++) {  
        int u, v;  
        scanf("%d %d", &u, &v);  
        graph[u][v] = 1;  
    }
```

```
    memset(matchR, 0, sizeof(matchR));
```

```
    for (int u = 1; u <= m; u++) {  
        memset(visited, 0, sizeof(visited));  
        bpm(u);  
    }
```

```
    int found = 0;
```



```

for (int u = 1; u <= m; u++) {
    for (int v = 1; v <= n; v++) {
        if (matchR[v] == u) {
            printf("(%d, %d)\n", u, v);
            found = 1;
        }
    }
}

if (!found)
    printf("No matching found");

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Alex needs to pair students from two different classes for a project, aiming to maximize the number of compatible pairs. Each student can be paired with only one student from the other class. The compatibility between students is represented as a binary adjacency matrix.

Alex wants to write a program to find the maximum number of compatible student pairs that can be formed.

Input Format

The first line contains an integer n , representing the number of students in each group.

The second line contains an integer m , representing the number of students in the second group.

The next n lines contain m space-separated integers each, representing the adjacency matrix of student compatibility.

Output Format

The single line containing the phrase "Maximum Bipartite Matching: x " where x is an integer representing the maximum number of compatible pairs that can be formed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

3

1 1 0

1 0 1

0 1 0

1 0 0

Output: Maximum Bipartite Matching: 3

Answer

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int n, m;
```

```
int graph[105][105];
```

```
int matchR[105];
```

```
int visited[105];
```

```
int bpm(int u) {
```

```
    for (int v = 0; v < m; v++) {
```

```
        if (graph[u][v] && !visited[v]) {
```

```
            visited[v] = 1;
```

```
            if (matchR[v] == -1 || bpm(matchR[v])) {
```

```
                matchR[v] = u;
```

```
                return 1;
```

```
        }
```

```
    }
```

```
}
```

```
    return 0;
```

```
}
```

```
int main() {
```

```
    scanf("%d", &n);
```

```
    scanf("%d", &m);
```

```
    for (int i = 0; i < n; i++)
```

```

for (int j = 0; j < m; j++)
    scanf("%d", &graph[i][j]);

for (int i = 0; i < m; i++)
    matchR[i] = -1;

int result = 0;

for (int u = 0; u < n; u++) {
    memset(visited, 0, sizeof(visited));
    if (bpm(u))
        result++;
}

printf("Maximum Bipartite Matching: %d", result);

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement:

John is developing a recommendation system for a social networking platform. The system suggests potential friends to users based on their interests and mutual connections. To enhance the user experience, you want to ensure that the recommendation algorithm considers the removal of certain users from the network due to various reasons such as account deletion or user activity.

However, he still wants to maintain effective friend suggestions even after removing these users. John plan to integrate a functionality that calculates the maximum matching in the bipartite graph representing the user network after removing specific users and their connections.

Help him to achieve the task.

Input Format

Enter the number of vertices in partition

Enter the number of vertices in partition

Enter the adjacency matrix representation of the bipartite graph

Enter the vertex to remove

Output Format

Maximum matching after removing vertex

Sample Test Case

Input: 3

2

0 1

1 0

0 0

1

Output: Maximum matching after removing vertex 1: 1

Answer

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int n, m;
```

```
int graph[15][15];
```

```
int matchR[15];
```

```
int visited[15];
```

```
int bpm(int u) {
```

```
    for (int v = 0; v < m; v++) {
```

```
        if (graph[u][v] && !visited[v]) {
```

```
            visited[v] = 1;
```

```
            if (matchR[v] == -1 || bpm(matchR[v])) {
```

```
                matchR[v] = u;
```

```
                return 1;
```

```
            }
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

```
int main() {
```

```

scanf("%d", &n);
scanf("%d", &m);

for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
        scanf("%d", &graph[i][j]);

int rem;
scanf("%d", &rem);

if (rem >= 0 && rem < n) {
    for (int j = 0; j < m; j++)
        graph[rem][j] = 0;
}

for (int j = 0; j < m; j++)
    matchR[j] = -1;

int result = 0;

for (int i = 0; i < n; i++) {
    if (i == rem) continue;
    memset(visited, 0, sizeof(visited));
    if (bpm(i))
        result++;
}

printf("Maximum matching after removing vertex %d: %d", rem, result);

return 0;
}

```

Status : Correct

Marks : 10/10